Lecture #2: Build LLm with pytorch.

#1: All primitives to train a model/
#2: bottom-up ~~from~~ [tensors to model to optimizer] → training loop
#3: efficiency (use of ~~t~~ resources) { memory (GB), compute (FLOPs) }

⇓

(15 TB) Num of Tokens ↗

(70B) model size ⇐⇒ ⇑ compute ~~1000~~ 1024 (~~8~~ H100 s)

example:

total flops: $\underline{6} * 70^{e9} * 15\ e12$ ↓ num of tokens, ↓ num of parameter.

⇓

H100_flop_per_sec == 1979e12/2.

⇓

mfu = 0.5

⇓

flops_per_day = h100_flop_per_sec * mfu * $\underline{1024}$ * $\underline{60 * 60 * 24}$ (Num of GPU ⇑, dayxsec ~~days for~~ secs ⇑)

⇓

days = total_flops / flops_per_day

①. how many hours to train a model?

②: what's the largest model that you can train on ...?

32-float / tensor. working

tensor-basic : / tensor. working

weights / parameters / graidences / acelvations / optimizer ~~op~~ state

model state /

[matrix]

sign 1 exponent 23 fraction

32-float (By default) :

16-float (half precision)

BF 16 float : Brain Floating = dynamic ~~ang~~ range

same memory as FP16 with as FP32

FP-8 : E4M3 and E5M2

=> ~~try~~ mix-precision training

- FP32 : inotreen
- FP16 / BF16 : unseable / overflow / underflow

e.g. FNN = backward path / forward path => FP8 / FP16

Transformer: Attention: caculation => FP32.

Computing accounting

CPU <=> GPU [SM SM SM] streaming multiprocessing